Deep Learning-Based Intelligent Visual Diagnosis of Tomato Yellow Leaf Curl Virus Using Leaf Images

Submitted in partial fulfillment of the requirements for the award of degree of

[DEGREE / PROGRAMME NAME]
IN
[BRANCH / SPECIALISATION]

[Institution logo]

Supervisor:
[Supervisor name and ID]
[Designation]
Co-Supervisor (if any):
[Name and ID]
[Designation]

Submitted By:
Ritesh Dwivedi
UID: [UID]

DEPARTMENT OF AGRICULTURAL SCIENCES
[University / Institute name], Chandigarh

November 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my supervisor, **[Supervisor name]**, [designation], and my co-supervisor, **[Co-supervisor name]**, [designation], [Department], [Institution], for their valuable guidance, constant encouragement and constructive suggestions throughout this work. Their support helped me shape the research problem, objectives and methodology presented in this synopsis.

I am thankful to the Head of the Department and all faculty members for providing the facilities and the academic environment needed to carry out this research. I also thank the evaluation committee for their useful comments during the review phases.

Ritesh Dwivedi
UID: [UID]

CERTIFICATE

This is to certify that the dissertation entitled "**Deep Learning-Based Intelligent Visual Diagnosis of Tomato Yellow Leaf Curl Virus Using Leaf Images**", Synopsis Viva-Voce examination of **Ritesh Dwivedi**, UID [UID], has been held on _____ and his case for the completion of eligibility for the award of the degree of [Degree / Programme name].

_____

Signature of Supervisor
[Supervisor name]
[Designation]
EID: [ID]

_____

Signature of Co-Supervisor
[Co-supervisor name]
[Designation]
EID: [ID]

_____

Signature of Examiner
[Examiner name]
EID: [ID]

CONTENTS

Acknowledgement ..... **ii**

Certificate ..... **iii**

List of Figures ..... **vi**

List of Tables ..... **vii**

List of Abbreviations ..... **viii**

CHAPTER 1: INTRODUCTION **1**

1.1 Background 1

1.2 Diagnostic Techniques for TYLCV 1

1.3 Deep Learning for Leaf-Image Diagnosis 2

1.4 Calibration and Explainability 3

1.5 Applications of the Research Area 3

1.6 Motivation 3

1.7 Organisation of the Report 3

CHAPTER 2: LITERATURE REVIEW **4**

2.1 Review Methodology 4

2.2 Surveys of Deep Learning in Plant Disease Diagnosis 4

2.3 Tomato Leaf Disease Classification 4

2.4 Dataset Bias and Generalisation to the Field 5

2.5 Detection, Segmentation and Severity Estimation 5

2.6 Lightweight Models, Calibration and Explainability 5

2.7 TYLCV Biology and Management Background 5

2.8 Comparative Analysis of Reviewed Literature 6

2.9 Key Findings from the Literature 6

CHAPTER 3: RESEARCH GAPS **8**

CHAPTER 4: PROBLEM FORMULATION **10**

4.1 Formal Formulation 10

CHAPTER 5: OBJECTIVES **11**

CHAPTER 6: METHODOLOGY **12**

6.1 Overall Framework 12

6.2 Step 1 – Dataset 12

6.3 Step 2 – Pre-processing and Splitting 13

6.4 Baseline Methods for Comparison 13

6.5 Step 3 – Proposed Model 14

6.6 Step 4 – Calibration, Explanation and Decision Support 14

6.7 Step 5 – Evaluation Protocol and Metrics 15

6.8 Tools and Experimental Setup 16

CHAPTER 7: PROOF OF WORK **17**

7.1 Implementation and Project Structure....17
7.2 Testing the Code....18
7.3 Result 1 – Audit of the Available TYLCV Image Data....18
7.4 Result 2 – Decision-Support Modules on a Simulated Benchmark....19
7.5 Result 3 – Status of the Image-Diagnosis Pipeline....22
7.6 Challenges and Risks....22
7.7 Plan for Remaining Work....23
CHAPTER 8: CONCLUSION....24
REFERENCES....25
ANNEXURE I: PLAGIARISM REPORT....28

LIST OF FIGURES

Figure 1.1: Reference pipeline for leaf-image TYLCV screening: capture, quality check, pretrained backbone, calibrated classifier, explanation and decision support.....2

Figure 6.1: Proposed methodology: what happens to the data at each step, and where decision support is used.....12

Figure 7.1: Project folder structure of the prototype.....17

Figure 7.2: Console output of the automated tests.....18

Figure 7.3: Audit of the public TYLCV image folder: source composition and label coverage.....19

Figure 7.4: ROC and precision–recall curves of the risk models on held-out simulated fields.....20

Figure 7.5: Risk surfaces: ground truth, v1 heuristic and v2 calibrated model (heat-maps).....20

Figure 7.6: Recombinant detection sensitivity by parent divergence and fragment length (heat-maps)21

Figure 7.7: Overall comparison of previous approaches, v1 and v2 on the simulated benchmark (radar chart).....22

LIST OF TABLES

Table 1.1: Main diagnostic techniques for TYLCV and their characteristics....2
Table 2.1: Typical benchmark-style approach and the proposed work....4
Table 2.2: Comparative analysis of reviewed literature....6
Table 3.1: Research gap analysis matrix....9
Table 6.1: Dataset summary....13
Table 6.2: Baseline methods planned for fair comparison....13
Table 6.3: Classifier configuration....14
Table 6.4: Decision-support modules of the TYLCV-Guard prototype....15
Table 6.5: Evaluation protocols....15
Table 6.6: Tools and justification....16
Table 7.1: Audit of the Kaggle TYLCV image folder....19
Table 7.2: Outbreak prediction on simulated fields (outbreak = incidence ≥ 30%; MAE of incidence, lower is better)....20
Table 7.3: Other modules on simulated data....21
Table 7.4: Schedule of remaining work....23

LIST OF ABBREVIATIONS

AUC	Area Under the (ROC) Curve
CAM	Class Activation Map
CNN	Convolutional Neural Network
DL	Deep Learning
ECE	Expected Calibration Error
F1	Harmonic mean of precision and recall
FPR	False Positive Rate
Grad-CAM	Gradient-weighted Class Activation Mapping
IPM	Integrated Pest Management
MAE	Mean Absolute Error
ML	Machine Learning
MoA	Mode of Action (insecticide group)
PCR	Polymerase Chain Reaction
PR	Precision-Recall
qPCR	Quantitative PCR
RGB	Red-Green-Blue (colour imaging)
ROC	Receiver Operating Characteristic
TPR	True Positive Rate
TYLCD	Tomato Yellow Leaf Curl Disease
TYLCV	Tomato Yellow Leaf Curl Virus
ViT	Vision Transformer
YOLO	You Only Look Once (object detector)

CHAPTER 1
INTRODUCTION

1.1 Background

Tomato (*Solanum lycopersicum*) is one of the most widely grown vegetable crops in the world, and its production is limited by many diseases. One of the most damaging is **tomato yellow leaf curl disease (TYLCD)**, caused by the begomovirus **tomato yellow leaf curl virus (TYLCV)**. Infected plants show leaflets that curl upward and inward, yellowing at the leaf margins, short internodes and a dwarfed, bushy habit, and they set few flowers and fruits. When plants are infected early, almost no marketable fruit is produced [1].

TYLCV is carried from plant to plant by the whitefly *Bemisia tabaci*, which takes up the virus while feeding and transmits it in a persistent way [2]. The virus has also been shown to be seed-transmissible in tomato [3]. No cure exists for an infected plant, so control depends on keeping whiteflies away, growing resistant varieties and removing infected plants early [1], [4], [5]. Finding infected plants early is therefore the first step of control. In practice this is done by trained people who walk the field and look at the leaves, and this does not scale to large farms or to farmers without training.

Cameras on phones are cheap and widespread, and deep learning has become good at recognising patterns in photographs. This has led to a large body of work on diagnosing plant diseases from leaf images [6], [7], [8]. This project asks a practical question: *how far can a deep-learning model that looks at a photograph of a tomato leaf be trusted to screen for TYLCV, and what does a trustworthy system need beyond the model itself?*

1.2 Diagnostic Techniques for TYLCV

Diagnosis can be made at several levels. Table 1.1 gives a short overview. The important point is that visual symptoms are only a screening signal: similar curling and yellowing can be caused by other begomoviruses, nutrient disorders or herbicide injury, so laboratory tests remain the confirmation [1].

Table 1.1: Main diagnostic techniques for TYLCV and their characteristics

Technique	Principle	Resources needed	Typical issues
Visual scouting	Trained observer recognises curling, yellowing, stunting	Trained staff, time	Subjective; scarce skill; look-alike disorders
PCR / qPCR	Detects viral DNA in leaf tissue	Laboratory, reagents	Cost and delay; not usable plant-by-plant in the field
RGB image + deep learning	Model learns the visual pattern from labelled photos	Phone or camera; labelled data; compute	Needs good data; drops in new conditions [6], [9]
Hyperspectral / thermal imaging	Detects physiological change, possibly before symptoms	Special sensors, calibration	Cost; mainly research use [10]
Image + IoT / weather context	Combines photos with sensor and environment data	Sensors, integration	Complex; data quality [11]

1.3 Deep Learning for Leaf-Image Diagnosis

A **convolutional neural network (CNN)** learns small visual filters that respond to edges, textures and colours, and stacks them so that deeper layers respond to larger patterns such as lesions or curled leaf shapes. Training a deep network from scratch needs very large datasets, so the usual approach is **transfer learning**: start from a network trained on millions of everyday photographs (ImageNet) and adapt it to leaf images [12], [13]. Common backbones are residual networks [14], the EfficientNet family [15] and small mobile-friendly networks such as MobileNetV3 [16]. Vision transformers split an image into patches and have become a strong alternative [17], [18]. When several leaves or symptoms appear in one photo, object detectors such as Faster R-CNN and YOLO draw boxes around them [19], [20]. Figure 1.1 shows the kind of pipeline used in this work.

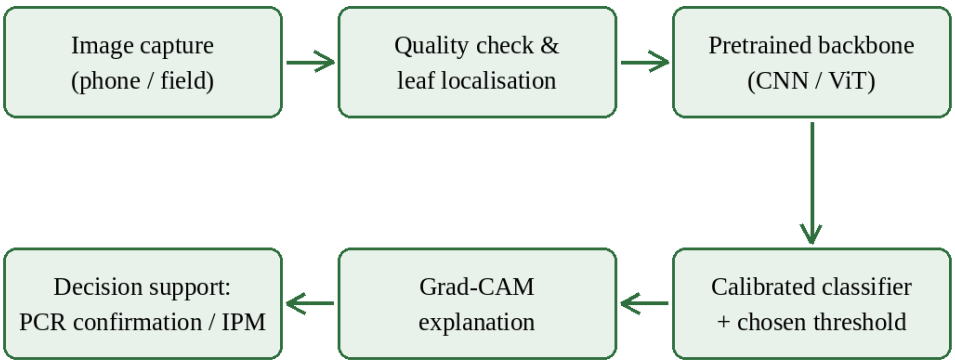


Figure 1.1: Reference pipeline for leaf-image TYLCV screening: capture, quality check, pretrained backbone, calibrated classifier, explanation and decision support

1.4 Calibration and Explainability

A model that outputs “92% TYLCV” is only useful if that number means what it says. Modern deep networks tend to be over-confident, and a simple fix is **temperature scaling** on a validation set [21]. The second need is explanation. **Grad-CAM** produces a heat-map showing which parts of the image drove the decision [22]. This matters because networks sometimes learn a **shortcut**, such as the background of the photo, instead of the disease [23].

1.5 Applications of the Research Area

- **Early screening on the farm:** a phone photo gives a quick yes/no flag that tells the grower which plants to send for PCR confirmation.
- **Scouting support:** field workers with little training can record symptomatic plants and locations for later action.
- **Disease surveillance:** geo-tagged flags can reveal where virus pressure is rising, together with whitefly counts and weather.
- **Teaching and extension:** heat-maps show students and farmers which leaf features indicate TYLCV.

1.6 Motivation

Three observations motivated this project. First, models reach above 99% accuracy on the popular PlantVillage benchmark [6], [24], yet one of the earliest studies reported that accuracy fell to about 31% on images taken under other conditions [6]. Second, the backgrounds of benchmark photos can leak the class label [25]. Third, when I examined a public TYLCV image set (Chapter 7) it contained only positive images, mostly lab-style photos and some near-duplicate pairs, so it could not support an honest estimate of accuracy. A trustworthy TYLCV screening tool needs better data, stricter evaluation and a link to what the grower does next (vector control, resistant varieties, laboratory confirmation).

1.7 Organisation of the Report

Chapter 2 reviews the literature. Chapter 3 lists the research gaps. Chapter 4 formulates the problem and Chapter 5 states the objectives. Chapter 6 describes the methodology. Chapter 7 presents the proof of work: the implemented prototype, its tests, a dataset audit, benchmark results and the plan for remaining work. Chapter 8 concludes. References and the plagiarism-report annexure follow.

CHAPTER 2
LITERATURE REVIEW

2.1 Review Methodology

I searched for studies on image-based plant disease diagnosis, tomato leaf disease classification, dataset bias and generalisation, detection and severity estimation, lightweight deployment, calibration and explainability, and on TYLCV biology and management. I concentrated on work from about 2016 to 2026 and on tomato. Studies were grouped by theme and compared qualitatively, because they differ in datasets, architectures and evaluation methods, which prevents a formal meta-analysis. Table 2.1 summarises how the proposed work differs from the typical benchmark-style approach, and Table 2.2 compares the studies.

Table 2.1: Typical benchmark-style approach and the proposed work

Aspect	Typical benchmark-style approach [6], [24]	Proposed work
Data	Lab-background leaf photos, one source	Lab, phone and field photos, several sources
Negatives	Other crop-disease classes	Healthy leaves plus look-alike conditions
Splitting	Random split of images	Near-duplicate-aware group split
Testing	Held-out images of the same kind	External field photos from other devices and sites
Outputs	Class label, accuracy	Calibrated probability, threshold, heat-map
Context	Image only	Linked to risk, resistance and vector-management modules

2.2 Surveys of Deep Learning in Plant Disease Diagnosis

Several surveys map the field. Kamilaris and Prenafeta-Boldú reviewed deep learning across agriculture [7]. Reviews focused on plant diseases cover architectures, datasets and open issues [26], [27], [8], [28], and a survey with practical recommendations stresses the tools needed for real deployment [29]. Reviews that include sensing and IoT show a move toward multi-source systems [11]. Across them, the recurring message is that benchmark accuracy is high but evidence of field performance and deployment is limited.

2.3 Tomato Leaf Disease Classification

The first large studies fine-tuned general networks on leaf photographs [13], [6]. A broad comparison of architectures reported success rates above 99% on an open database [24], and a

comparison of fine-tuned networks found that several perform similarly well on PlantVillage [12]. Tomato-specific studies compared standard CNNs [30], [31], proposed lightweight CNNs [32], and added attention modules to residual networks [33], [34]. TYLCV is usually just one class among several in these studies, most of which use the PlantVillage tomato classes.

2.4 Dataset Bias and Generalisation to the Field

The central finding of the literature is the gap between laboratory and field. In the first large PlantVillage study, accuracy that was very high on held-out lab images dropped to roughly 31% on images from other conditions [6]. Barbedo showed that dataset size and, above all, variety of conditions decide whether transfer learning generalises [9], and that classifying individual lesions and spots can help [35]. Studies using smartphone images captured in real fields found that field-captured training images are needed [36], [37]. Lee and co-workers argued for training strategies that reflect real variation [38]. Noyan showed that PlantVillage backgrounds alone carry enough information to classify images well above chance [25], an example of the general problem of shortcut learning [23]. The PlantDoc dataset was assembled to provide harder, in-the-wild images [39].

2.5 Detection, Segmentation and Severity Estimation

Object detectors localise symptoms in a scene. A real-time detector for tomato diseases and pests used region-based detectors with several backbones on field images [40], and a greenhouse system combined detection with severity estimation [41]. Detection needs boxes or masks, which are expensive to label; I did not find a TYLCV-specific annotated set of this kind.

2.6 Lightweight Models, Calibration and Explainability

Field use favours small, fast models [16], [29]. Calibrated probabilities are rarely reported in plant-disease studies, although temperature scaling is simple and effective [21]. Saliency maps were used early to inspect tomato classifiers [31] and Grad-CAM is now the standard explanation tool [22]. Saliency is an aid to spot shortcuts, not proof of correctness.

2.7 TYLCV Biology and Management Background

An image model is only one part of TYLCV control. Resistance genes (*Ty-1* to *Ty-6*) introgressed from wild tomato relatives are widely used but are not fully durable [4], [42], [43]. Recombinant viruses and heat can weaken resistance, and geographic spread continues

[44], [45], [46]. Whitefly control relies on exclusion nets, mulches and insecticides, and insecticide resistance in *B. tabaci* is a growing problem [47], [48]. Climate change is expected to alter pest pressure [49]. These facts motivated linking the image model to decision-support modules (Chapter 6).

2.8 Comparative Analysis of Reviewed Literature

Table 2.2: Comparative analysis of reviewed literature

Ref.	Focus	Method and data	Main finding
[6]	Plant disease classification	AlexNet/GoogLeNet transfer learning; PlantVillage (54,306 images, 38 classes)	99.35% on held-out lab images; about 31% on images from other conditions
[24]	Disease detection and diagnosis	Several CNN architectures; open database, 58 classes	Best model reached 99.53% success on the database
[31]	Tomato disease classification and visualisation	Pretrained CNNs on tomato classes; saliency maps	Visualisation localises symptoms and aids interpretation
[12]	Fine-tuning comparison	Several CNNs fine-tuned on PlantVillage	Fine-tuned networks perform similarly well on this benchmark
[9]	Dataset size and variety	Transfer learning on varied datasets	Variety of conditions matters more than adding similar images
[35]	Lesions and spots	Classification of individual lesions	Can improve robustness over whole-leaf classification
[36]	Smartphone images in the wild	CNNs on mobile-captured field images	Field-captured training images needed for usable accuracy
[37]	Cassava disease	Transfer learning on field images	Shows feasibility with images from real fields
[40]	Tomato disease and pest detection	Region-based detectors, several backbones, field images	Real-time detection of several tomato problems
[41]	Greenhouse tomato diseases	Detection and severity estimation	Combines localisation with severity grading
[33]	Tomato leaf disease	Attention-embedded residual CNN; PlantVillage tomato	Attention helps on the benchmark
[34]	Tomato leaf disease	Residual network with attention module	Reports improved benchmark accuracy
[39]	Visual disease detection dataset	In-the-wild images with annotations	Provides harder test conditions
[25]	Dataset bias	Background-only classification experiment	PlantVillage backgrounds carry class information
[23]	Shortcut learning	General analysis	Networks may exploit spurious cues
[21], [22]	Calibration and explanation	Temperature scaling; Grad-CAM	Reliable probabilities and visual explanations

2.9 Key Findings from the Literature

- Accuracy above 95–99% on lab-background benchmarks is common [6], [24], [12], but it measures capacity, not field performance.

- Generalisation depends on data variety and realistic evaluation more than on the choice of architecture [9], [38].
- Backgrounds can leak labels [25], [23], so explanation checks and external tests are needed.
- TYLCV is usually one class among many; I did not find a field-validated TYLCV study with molecularly confirmed ground truth.
- Calibration, thresholds and links to management decisions are rarely reported [21], [29].

CHAPTER 3
RESEARCH GAPS

After studying the literature in Table 2.2 and auditing one public TYLCV image set (Chapter 7), six research gaps stand out. Each gap is listed with the evidence behind it.

Gap 1: Missing negatives and look-alike conditions.

- The public TYLCV folder I audited contains 100 images, all TYLCV-positive; accuracy, specificity and calibration cannot be estimated from it.
- Public multi-class sets mostly contain fungal or bacterial spots, which look very different from a viral curl [31], [30]; look-alikes such as other begomoviruses, nutrient disorders and herbicide injury are rare.

Gap 2: Lab-background shortcuts and weak field generalisation.

- Accuracy fell from very high to about 31% when conditions changed [6].
- PlantVillage backgrounds alone carry class information [25]; 91 of the 100 images in the audited folder have PlantVillage-style names and the same 256×256 format.

Gap 3: Leakage through near-duplicate images.

- The audit found two pairs of the same photograph stored at different resolutions; a random split would put one copy in training and one in testing.
- Most reviewed studies report a single accuracy on a random split without group-aware splitting.

Gap 4: No calibrated probabilities or validated thresholds.

- Accuracy is reported, but calibration and the choice of operating threshold are rarely examined [21].

Gap 5: Image diagnosis is disconnected from management decisions.

- Reviews call for tools that link diagnosis to recommendations [29], [11]; TYLCV control depends on vector pressure, resistance durability and insecticide resistance [4], [47].

Gap 6: Weak ground truth and reporting.

- Public labels are usually visual, not PCR-confirmed, and most studies report no confidence intervals or external validation.

Table 3.1: Research gap analysis matrix

Research area	Current limitation	What this work does
Data	Positive-only or lab-style sets (Gap 1, 2)	Audits data; requires negatives and field photos; group-aware design
Evaluation	Random split; near-duplicates leak (Gap 3)	Duplicate-aware split; external field set; confidence intervals
Model output	Accuracy only (Gap 4)	Temperature-scaled probability; threshold chosen on validation data
Explanation	Rarely checked for shortcuts (Gap 2)	Grad-CAM and background-only control
Decision context	Image only (Gap 5)	TYLCV-Guard risk, resistance, rotation and surveillance modules
Ground truth	Visual labels (Gap 6)	Protocol requires PCR-confirmed field subset

CHAPTER 4
PROBLEM FORMULATION

4.1 Formal Formulation

Input. An RGB photograph x of a tomato leaf (or leaflet) taken with a phone or camera, optionally with context c (temperature, whitefly density, crop stage, resistance loci of the cultivar).

Output. A calibrated probability $\hat{p}(x) = P(\text{TYLCV} \mid x)$, a binary screening decision $\hat{y} = 1[\hat{p} \geq \tau]$ and a heat-map $H(x)$ that explains the decision. With context, a risk index $r(c)$ is reported next to the image result.

Learning task. Given labelled images with labels $y \in \{0, 1\}$ (0 = not TYLCV, 1 = TYLCV), train a classifier with parameters θ by minimising the class-weighted cross-entropy:

$$L(\theta) = -(1/N) \Sigma_i \, w_{y_i} \left[y_i \log p_\theta(x_i) + (1 - y_i) \log(1 - p_\theta(x_i)) \right] \quad (4.1)$$

where the weights w_0, w_1 compensate for class imbalance.

Calibration task. On validation logits z, find a temperature $T > 0$ that minimises the negative log-likelihood of the scaled probabilities, and a threshold τ that maximises F1 on validation data only:

$$\hat{p} = \text{softmax}(z / T), \quad T^\star = \arg\min_T \text{NLL}_{\text{val}}(z / T), \quad \tau^\star = \arg\max_\tau \text{F1}_{\text{val}}(\hat{p} \geq \tau) \quad (4.2)$$

Evaluation task. On a test set that shares no near-duplicate group with the training data, and on an external field set from other sites and devices, report sensitivity, specificity, ROC-AUC and the expected calibration error:

$$\text{ECE} = \Sigma_b \, (|B_b| / N) \cdot | \, \text{acc}(B_b) - \text{conf}(B_b) \, | \quad (4.3)$$

where the predictions are grouped into bins B_b. Group-aware splitting assigns every image to a near-duplicate group $g(x)$ and requires that no group appears in more than one of the training, validation and test sets.

CHAPTER 5
OBJECTIVES

To address the research gaps and solve the problem formulated in Chapter 4, the main objectives of this research are:

1. To study and analyse existing approaches for deep-learning-based leaf-image diagnosis of tomato diseases, with emphasis on TYLCV, and on the management measures that follow a diagnosis.

2. To develop and implement a leaf-image TYLCV screening pipeline with calibrated outputs and visual explanations, together with decision-support modules that connect diagnosis to risk, resistance and vector management.

3. To compare and validate the proposed approach against baseline methods under realistic conditions, using duplicate-aware splits and external field photographs.

CHAPTER 6
METHODOLOGY

6.1 Overall Framework

Figure 6.1 shows the whole method on one page. Read it from top to bottom: leaf photographs go in at Step 1, and a calibrated probability with a heat-map comes out at the bottom. The purple box on the right is the decision-support part (TYLCV-Guard), which supplies risk context to the decision. The grey box lists the methods every result is compared against.

Objective 1 was covered by the literature review and gap analysis in Chapters 2 and 3. Steps 1 to 4 and the decision-support box make up Objective 2 (develop and implement). Step 5 is Objective 3 (evaluate and validate).

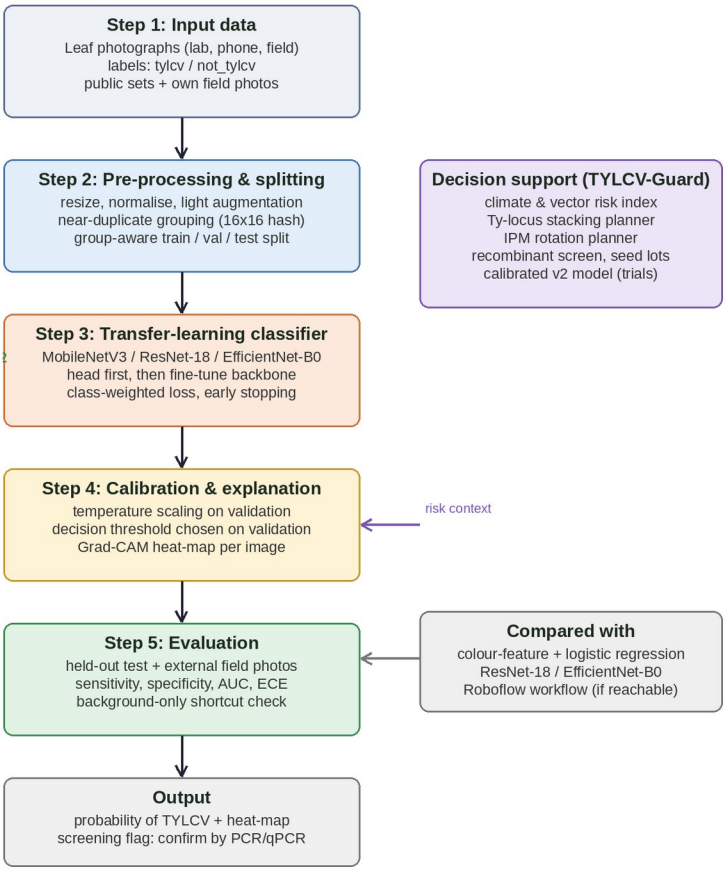


Figure 6.1: Proposed methodology: what happens to the data at each step, and where decision support is used

6.2 Step 1 – Dataset

Table 6.1 lists the data sources. One public Kaggle image folder of TYLCV leaves is available and has been audited (Chapter 7); a second public set is available through a

Roboflow Universe project [50], which was not accessible from my development environment. A labelled dataset with healthy leaves and other diseases, and a set of field photos, are required before training and are the next stage of work.

Table 6.1: Dataset summary

Source	Description	Status
Kaggle TYLCV folder [51]	100 JPEG images of TYLCV leaves/plants: 91 lab-style (256×256), 7 phone photos (3000×4000), 2 field photos (960×1280); no negatives	Audited
Roboflow Universe TYLCV dataset [50]	Public dataset; also reachable through a Roboflow workflow API	Not audited; API not reachable from the dev environment
Labelled tomato leaf set with negatives	Healthy leaves and other tomato diseases (e.g. PlantVillage tomato classes [6])	To be obtained; licence to be checked
Field photographs	Phone photos of healthy, TYLCV and look-alike plants from several sites	To be collected
Field trials (TYLCV-Guard)	Trial records: site, loci, temperature, whitefly density, stage, incidence	Logging interface implemented

6.3 Step 2 – Pre-processing and Splitting

Resizing and normalisation. Images are resized and centre-cropped to 224 × 224 pixels and normalised with ImageNet statistics. **Augmentation.** Random crops, flips, rotation and *small* brightness, contrast, saturation and hue changes are used; colour changes are kept small because yellowing is the diagnostic signal.

Near-duplicate grouping. Each image is converted to a 16 × 16 grey average hash (256 bits). Images whose hashes differ in at most 12 bits are placed in one group. This threshold was checked on the audited folder, where it linked the true duplicate pairs and no unrelated images. A coarser 8 × 8 hash was tried first and wrongly merged unrelated leaves on plain backgrounds, so it was replaced. **Group-aware split.** Groups are assigned, class by class, to training (70%), validation (15%) and test (15%) sets, so no group crosses sets.

6.4 Baseline Methods for Comparison

Table 6.2: Baseline methods planned for fair comparison

Baseline	Description
Colour-feature + logistic regression	Mean/variance colour statistics and simple texture features; shows how much a trivial model achieves
Background-only classifier	Trained on image borders only; detects shortcut learning [25]
ResNet-18 [14]	Standard fine-tuned residual network
EfficientNet-B0 [15]	Compound-scaled CNN
Roboflow workflow	Existing hosted model on the same test images, if reachable [50]

6.5 Step 3 – Proposed Model

The classifier is a transfer-learning CNN, MobileNetV3-Small by default [16] (ResNet-18 and EfficientNet-B0 are available). Training has two phases: first only the new classification head is trained, then the whole network is fine-tuned at a ten-times lower learning rate [12]. Table 6.3 gives the configuration. If pretrained weights cannot be downloaded the code warns loudly and trains from random initialisation, which is expected to perform much worse.

Table 6.3: Classifier configuration

Item	Setting
Backbone	MobileNetV3-Small (default), ResNet-18, EfficientNet-B0; ImageNet-pretrained
Input	224 × 224 RGB, ImageNet normalisation
Head	Final linear layer replaced by 2 outputs (not_tylcv, tylcv)
Loss	Class-weighted cross-entropy (Equation 4.1)
Optimiser	AdamW, weight decay 1e-4; head lr 1e-3, then 1e-4 for the full network
Schedule	2 frozen-backbone epochs, then fine-tuning; early stopping on validation AUC (patience 4)
Seeds	Fixed seeds; at least three seeds planned for the final results

6.6 Step 4 – Calibration, Explanation and Decision Support

Calibration. Temperature scaling is fitted on validation logits and the decision threshold is chosen on validation data (Equation 4.2). **Explanation.** Grad-CAM heat-maps are produced for each prediction [22], and a background-only baseline checks for shortcuts.

Decision support (TYLCV-Guard). Besides the image model, I built a web application that connects a diagnosis to the actions that follow. Its modules are summarised in Table 6.4. They are advisory and based on assumptions taken from the literature; they are not validated against field data.

Table 6.4: Decision-support modules of the TYLCV-Guard prototype

Module	What it does	Basis
Climate and vector risk index	Daily and seasonal risk from temperature, whitefly density, crop stage, resistance and exclusion	Heuristic from literature [1], [49]
Resistance stacking planner	Scores combinations of Ty loci and suggests the next locus	Heuristic priors [4], [42], [43]
IPM rotation planner	Plans insecticide mode-of-action rotation; excludes failed groups; no product or dose advice	Resistance management [47], [48]
Sequence surveillance	k-mer classification and recombinant screen for user-supplied sequences	Genomic surveillance [44], [46]
Seed-lot register	Status of seed lots and a rule-of-three prevalence bound	Seed transmission [3]
Field-trial log and calibrated model	Logs trials; fits a logistic model of incidence from the data	Learns from data (needs 30+ trials)
Leaf scan	Image screening through a Roboflow workflow or a locally trained model	[50]

6.7 Step 5 – Evaluation Protocol and Metrics

Table 6.5: Evaluation protocols

Protocol	Purpose	Metrics
Duplicate-aware held-out test	Honest in-distribution estimate	Accuracy, sensitivity, specificity, F1, ROC-AUC, PR-AUC, confusion matrix
External field set	Measure domain shift (new device, background, site)	Sensitivity and specificity with 95% confidence intervals
Hard-negative set	Specificity against look-alike conditions	False-positive rate per condition
Calibration check	Are probabilities reliable?	ECE, reliability diagram [21]
Shortcut check	Does the model use leaf or background?	Background-only accuracy; Grad-CAM [22]
Repeated seeds	Training variability	Mean ± standard deviation over seeds

6.8 Tools and Experimental Setup

Table 6.6: Tools and justification

Tool	Use	Why
Python 3.13	All code	Standard for ML and data science
PyTorch, torchvision	Model training and inference	Pretrained backbones and Grad-CAM hooks
scikit-learn, NumPy, Matplotlib	Metrics and figures	Reproducible evaluation
SQLite + Python standard library	Web application and API	No external dependencies; easy to deploy
Roboflow inference SDK (optional)	Hosted workflow inference	Compare with an existing hosted model
unittest	Automated tests	34 tests for modules and pipeline

CHAPTER 7
PROOF OF WORK

A working prototype of the platform and the training pipeline has already been built. This chapter shows the code, how it was tested, an audit of the available TYLCV image data, benchmark results for the decision-support modules, and the plan for remaining work. **Important:** no image classifier has yet been trained or evaluated on real leaf photographs with negatives, because such data was not available; the results below are therefore about the platform, the data and the evaluation design, not about image-diagnosis accuracy.

7.1 Implementation and Project Structure

The prototype is written in Python as the package **tylcv-guard**. Figure 7.1 shows how it is organised. The main parts are:

- **app/risk.py, stacking.py, rotation.py:** the risk index, resistance stacking planner and insecticide rotation planner.
- **app/seqtools.py, calibrate.py, db.py, server.py:** sequence screening, the calibrated model, SQLite storage and the JSON API.
- **app/vision.py:** Leaf scan through a Roboflow workflow or a locally trained model; credentials are read only from environment variables.
- **dl/data.py, model.py, train.py, infer.py:** scanning, duplicate-aware splitting, backbones, Grad-CAM, training, evaluation and inference.
- **benchmark/ and tools/:** a simulator and benchmark producing 15 figures, and a dataset audit tool.

Project folder structure

```
tylcv-guard/
├── app/                 stdlib server + modules
│   ├── risk.py          climate & vector risk index
│   ├── stacking.py      Ty-locus stacking planner
│   ├── rotation.py      MoA rotation planner
│   ├── seqtools.py      k-mer classify, recombinant screen
│   ├── calibrate.py     calibrated v2 model (from trials)
│   ├── vision.py        Roboflow / local-model Leaf scan
│   ├── db.py, server.py SQLite storage, JSON API
├── dl/                  leaf-image deep learning
│   ├── data.py          scan, duplicate groups, splits
│   ├── model.py         backbones, Grad-CAM
│   └── train.py, infer.py   train/evaluate, predict
├── benchmark/           simulator, run.py, 15 figures
├── tools/               dataset audit, batch scan
├── static/index.html    web interface (8 tabs)
├── tests/               34 automated tests
└── README.md, Dockerfile, requirements-optional.txt
```

Figure 7.1: Project folder structure of the prototype

7.2 Testing the Code

Automated tests check each module and the pipeline: validation of inputs, monotonic behaviour of the risk index, locus stacking, rotation constraints, sequence classification and recombination screening, database operations, API endpoints, the vision integration with a mocked client, and the deep-learning pipeline. The deep-learning tests use *synthetic* images only: they show that data scanning, duplicate-aware splitting, training, evaluation, Grad-CAM and inference run end to end and that near-duplicates do not cross splits. They say nothing about accuracy on real leaves. All 34 tests pass (Figure 7.2).

Automated test run

```
$ python -m unittest discover -s tests -t . -v
test_bad_json ... ok
test_endpoints ... ok
test_pages_and_data ... ok
test_efficacy_rotation ... ok
test_fit_learns_direction ... ok
test_too_few_rows ... ok
test_v2_screen ... ok
test_seedlots ... ok
test_surveillance_flow ... ok
test_trials ... ok
...
test_sdk_error_is_sanitised ... ok
test_validation ... ok
test_workflow_called_with_config_and_cleanup ... ok
test_labels_and_split_without_leakage ... ok
Ran 34 tests in 22.610s
OK
```

Figure 7.2: Console output of the automated tests

During testing the first version of the duplicate hash (8 × 8 bits) merged unrelated synthetic images into very large groups and left the validation split empty; it was replaced by the stricter 16 × 16 hash described in Chapter 6.

7.3 Result 1 – Audit of the Available TYLCV Image Data

I audited the public Kaggle TYLCV folder [51] before using it. Table 7.1 and Figure 7.3 summarise the findings.

Table 7.1: Audit of the Kaggle TYLCV image folder

Item	Finding
Number of images	100 JPEG files, none corrupt
Dimensions	91 images 256×256; 7 images 3000×4000; 2 images 960×1280
Sources (by filename)	91 PlantVillage-style names; 7 phone-camera names; 2 timestamp names
Exact duplicates	None
Near-duplicates	Two pairs: the same photograph stored at different resolutions
Labels	All 100 images TYLCV-positive; no healthy or other-disease images
Consequence	Accuracy, specificity and calibration cannot be measured; only the 9 non-lab images test field generalisation

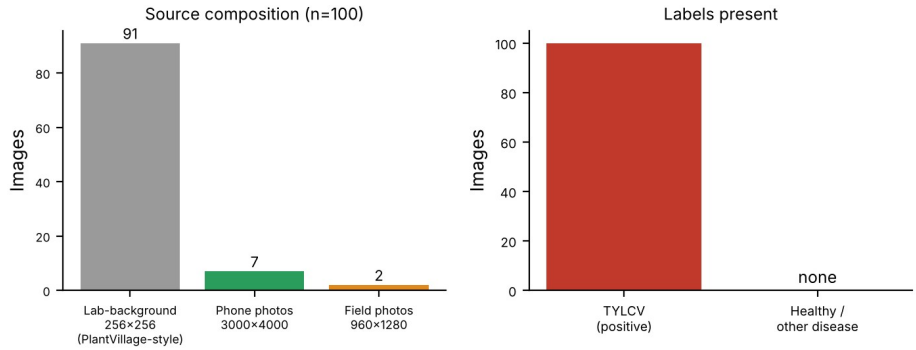


Figure 7.3: Audit of the public TYLCV image folder: source composition and label coverage

This result directly led to the design choices of Chapter 6: duplicate-aware splitting, a requirement for negatives and an external field test set. I make no accuracy claim for any image model on this folder.

7.4 Result 2 – Decision-Support Modules on a Simulated Benchmark

To test the decision-support modules I built a simulator of fields (temperature, whitefly density, crop stage, resistance loci, recombinant pressure, exclusion) with a ground-truth infection process that uses different functional forms from the modules. Previous approaches (temperature-only, whitefly-threshold and temperature × whitefly) were compared with the first heuristic model (v1) and a calibrated model (v2) learned from the simulated trials. **All numbers come from simulated data and show how the components behave under assumed rules; they are not evidence of real-world accuracy.** Table 7.2 and Figures 7.4–7.7 show the results.

Table 7.2: Outbreak prediction on simulated fields (outbreak = incidence ≥ 30%; MAE of incidence, lower is better)

Model	AUC	Avg. precision	MAE	ECE	AUC (hot, unseen)
Temp-only	0.635	0.222	0.126	0.069	0.666
Whitefly-threshold	0.736	0.314	0.125	0.041	0.732
Env-only (T×WF)	0.764	0.348	0.117	0.017	0.775
v1 (heuristic)	0.930	0.786	0.074	0.022	0.938
v2 (calibrated)	**0.980**	**0.897**	**0.038**	**0.003**	**0.982**

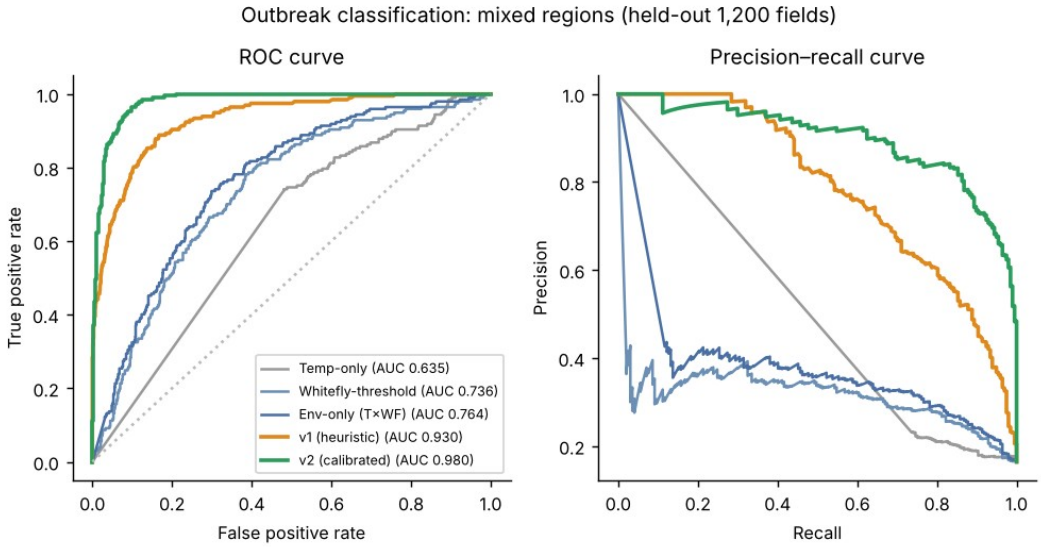


Figure 7.4: ROC and precision–recall curves of the risk models on held-out simulated fields

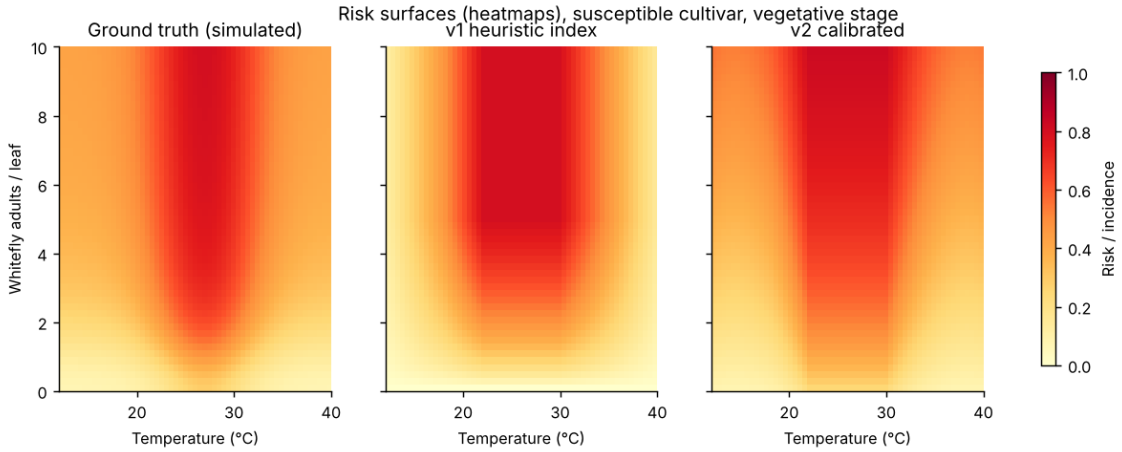


Figure 7.5: Risk surfaces: ground truth, v1 heuristic and v2 calibrated model (heat-maps)

The calibrated model matched the heuristic with about 30 simulated trials (AUC 0.924) and clearly exceeded it from about 60 (AUC 0.969), which shows why logging field trials matters. The advantage of v1 and v2 over the previous approaches comes mostly from including

resistance and crop stage, which the previous approaches ignore by construction; this comparison is therefore tilted in their favour.

Table 7.3: Other modules on simulated data

Module	Previous approach	v1	v2
Recombinant detection (mean sensitivity, ~1% false positives)	0.00	0.67	0.80
Rotation: mean efficacy, uniform resistance	0.04 (one group repeated)	0.39	0.39
Rotation: mean efficacy, two groups already compromised	0.03 (one group repeated)	0.35	0.35

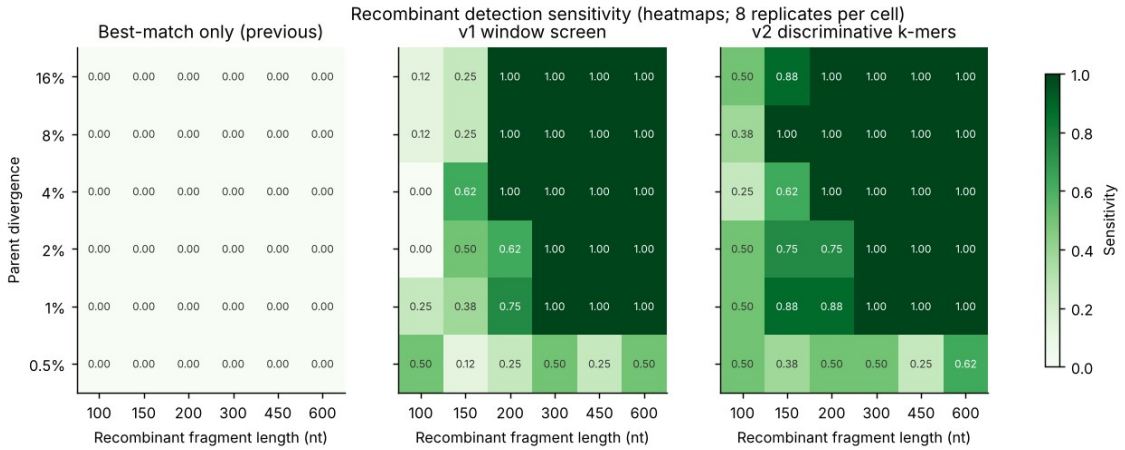


Figure 7.6: Recombinant detection sensitivity by parent divergence and fragment length (heat-maps)

Honest caveats: the rotation planners did **not** beat a fixed round-robin schedule in the toy model, and every strategy lost efficacy within about 20 sprays, so rotation alone is not sufficient; the recombinant screen fails at 0.5% divergence and for fragments of 100 nt or less; and the simulator shares its qualitative assumptions with v1 and v2.

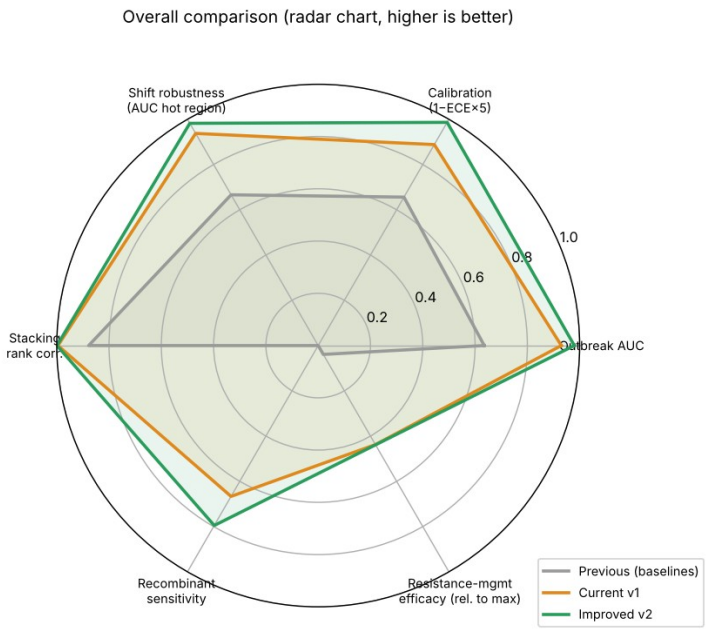


Figure 7.7: Overall comparison of previous approaches, v1 and v2 on the simulated benchmark (radar chart)

7.5 Result 3 – Status of the Image-Diagnosis Pipeline

The deep-learning pipeline is complete and tested on synthetic data, and the Leaf scan integration with the Roboflow workflow was tested against a mocked client because the hosted service was not reachable from the development environment. Real training was blocked by two things: the lack of a labelled set with negatives, and the lack of network access to download pretrained weights. These are listed as risks below. I have deliberately not reported any accuracy for an image model.

7.6 Challenges and Risks

- **Data:** negatives, look-alike conditions and field photos must be obtained; labels from public sets are visual, not PCR-confirmed.
- **Shortcut learning:** lab-background data may let the model learn the background; mitigated by external field tests and Grad-CAM.
- **Licences:** the licences of public datasets must be checked before use or redistribution.
- **Compute and weights:** training needs pretrained weights and, ideally, a GPU.
- **Overclaiming:** the simulated benchmark must not be presented as field accuracy; results from real data must be reported separately.

7.7 Plan for Remaining Work

Table 7.4: Schedule of remaining work

Period	Work	Output
Month 1	Obtain a labelled tomato leaf set with negatives and look-alikes; check licences; collect field photos	Dataset with documented sources
Month 2	Train baselines and MobileNetV3 / ResNet-18 / EfficientNet-B0 with several seeds; duplicate-aware splits	Test metrics with confidence intervals
Month 3	External field evaluation, background-only and Grad-CAM shortcut checks, calibration	Domain-shift and calibration results
Month 4	Integrate the trained model into Leaf scan; compare with the Roboflow workflow; field trial logging	Integrated prototype and comparison
Month 5	PCR-confirmed subset (if access is available); statistical tests	Validated results
Month 6	Thesis writing and submission	Final dissertation

CHAPTER 8
CONCLUSION

This synopsis looked at deep-learning-based diagnosis of tomato yellow leaf curl virus from leaf images. The literature review showed that models reach very high accuracy on lab-background benchmarks, but that this does not carry over to field images, that benchmark backgrounds can leak labels, that near-duplicate images can inflate results, and that calibration, thresholds and links to management decisions are rarely addressed. I did not find a field-validated TYLCV study with molecularly confirmed ground truth.

Three objectives were set: to study existing approaches and find their gaps; to develop and implement a screening pipeline with calibrated outputs, explanations and decision-support modules; and to evaluate it under realistic conditions. A working prototype of the platform and the training pipeline has been built and passes 34 automated tests. An audit of a public TYLCV image set showed that it contains only positive images, mostly lab-style photos and near-duplicate pairs, which is why the design uses duplicate-aware splits, negatives and external field tests. Decision-support modules were benchmarked on simulated data, with results that show where they help and where they do not. No accuracy has yet been measured for an image classifier on real data; the next stage will obtain labelled data with negatives, train and evaluate the models with several seeds, test them on external field photos and report honest confidence intervals.

REFERENCES

[1] A. Prasad, N. Sharma, G. Hari-Gowthem, M. Muthamilarasan, and M. Prasad, "Tomato yellow leaf curl virus: Impact, challenges, and management," *Trends Plant Sci.*, vol. 25, no. 9, pp. 897–911, 2020.

[2] H. Czosnek, A. Hariton-Shalev, I. Sobol, R. Gorovits, and M. Ghanim, "The incredible journey of begomoviruses in their whitefly vector," *Viruses*, vol. 9, no. 10, p. 273, 2017.

[3] E.-J. Kil *et al.*, "Tomato yellow leaf curl virus (TYLCV-IL): A seed-transmissible geminivirus in tomatoes," *Sci. Rep.*, vol. 6, Art. no. 19013, 2016.

[4] Z. Yan *et al.*, "Resistance to tomato yellow leaf curl virus in tomato germplasm," *Front. Plant Sci.*, vol. 9, Art. no. 1198, 2018.

[5] M. R. Rojas *et al.*, "World management of geminiviruses," *Annu. Rev. Phytopathol.*, vol. 56, pp. 637–677, 2018.

[6] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, Art. no. 1419, 2016.

[7] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Comput. Electron. Agric.*, vol. 147, pp. 70–90, 2018.

[8] A. Abade, P. A. Ferreira, and F. de Barros Vidal, "Plant diseases recognition on images using convolutional neural networks: A systematic review," *Comput. Electron. Agric.*, vol. 185, Art. no. 106125, 2021.

[9] J. G. A. Barbedo, "Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification," *Comput. Electron. Agric.*, vol. 153, pp. 46–53, 2018.

[10] A.-K. Mahlein, "Plant disease detection by imaging sensors – parallels and specific demands for precision agriculture and plant phenotyping," *Plant Dis.*, vol. 100, no. 2, pp. 241–251, 2016.

[11] M. Ouhami, A. Hafiane, Y. Es-Saady, M. El Hajji, and R. Canals, "Computer vision, IoT and data fusion for crop disease detection using machine learning: A survey and ongoing research," *Remote Sens.*, vol. 13, no. 13, p. 2486, 2021.

[12] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Comput. Electron. Agric.*, vol. 161, pp. 272–279, 2019.

[13] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, "Deep neural networks based recognition of plant diseases by leaf image classification," *Comput. Intell. Neurosci.*, vol. 2016, Art. no. 3289801, 2016.

[14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[15] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[16] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1314–1324.

[17] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[18] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 10012–10022.

[19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017.

[20] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[22] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[23] R. Geirhos *et al.*, "Shortcut learning in deep neural networks," *Nat. Mach. Intell.*, vol. 2, pp. 665–673, 2020.

[24] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Comput. Electron. Agric.*, vol. 145, pp. 311–318, 2018.

[25] M. A. Noyan, "Uncovering bias in the PlantVillage dataset," *arXiv:2206.04374*, 2022.

[26] M. H. Saleem, J. Potgieter, and K. M. Arif, "Plant disease detection and classification by deep learning," *Plants*, vol. 8, no. 11, p. 468, 2019.

[27] J. Liu and X. Wang, "Plant diseases and pests detection based on deep learning: A review," *Plant Methods*, vol. 17, Art. no. 22, 2021.

[28] M. Shoaib *et al.*, "An advanced deep learning models-based plant disease detection: A review of recent research," *Front. Plant Sci.*, vol. 14, Art. no. 1158933, 2023.

[29] A. Ahmad, D. Saraswat, and A. El Gamal, "A survey on using deep learning techniques for plant disease diagnosis and recommendations for development of appropriate tools," *Smart Agric. Technol.*, vol. 3, Art. no. 100083, 2023.

[30] K. Zhang, Q. Wu, A. Liu, and X. Meng, "Can deep learning identify tomato leaf disease?" *Adv. Multimedia*, vol. 2018, Art. no. 6710865, 2018.

[31] M. Brahimi, K. Boukhalfa, and A. Moussaoui, "Deep learning for tomato diseases: Classification and symptoms visualization," *Appl. Artif. Intell.*, vol. 31, no. 4, pp. 299–315, 2017.

[32] M. Agarwal, A. Singh, S. Arjaria, A. Sinha, and S. Gupta, "ToLeD: Tomato leaf disease detection using convolution neural network," *Procedia Comput. Sci.*, vol. 167, pp. 293–301, 2020.

[33] R. Karthik, M. Hariharan, S. Anand, P. Mathikshara, A. Johnson, and R. Menaka, "Attention embedded residual CNN for disease detection in tomato leaves," *Appl. Soft Comput.*, vol. 86, Art. no. 105933, 2020.

[34] S. Zhao, Y. Peng, J. Liu, and S. Wu, "Tomato leaf disease diagnosis based on improved convolution neural network by attention module," *Agriculture*, vol. 11, no. 7, p. 651, 2021.

[35] J. G. A. Barbedo, "Plant disease identification from individual lesions and spots using deep learning," *Biosyst. Eng.*, vol. 180, pp. 96–107, 2019.

[36] A. Picon *et al.*, "Deep convolutional neural networks for mobile capture device-based crop disease classification in the wild," *Comput. Electron. Agric.*, vol. 161, pp. 280–290, 2019.

[37] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. P. Hughes, "Deep learning for image-based cassava disease detection," *Front. Plant Sci.*, vol. 8, Art. no. 1852, 2017.

[38] S. H. Lee, H. Goëau, P. Bonnet, and A. Joly, "New perspectives on plant disease characterization based on deep learning," *Comput. Electron. Agric.*, vol. 170, Art. no. 105220, 2020.

[39] D. Singh *et al.*, "PlantDoc: A dataset for visual plant disease detection," in *Proc. ACM India Joint Int. Conf. Data Sci. Manage. Data (CoDS-COMAD)*, 2020, pp. 249–253.

[40] A. Fuentes, S. Yoon, S. C. Kim, and D. S. Park, "A robust deep-learning-based detector for real-time tomato plant diseases and pests recognition," *Sensors*, vol. 17, no. 9, p. 2022, 2017.

[41] P. Wspanialy and M. Moussa, "A detection and severity estimation system for generic diseases of tomato greenhouse plants," *Comput. Electron. Agric.*, vol. 178, Art. no. 105701, 2020.

[42] H. Yamaguchi *et al.*, "An NB-LRR gene, *TYNBS1*, is responsible for resistance mediated by the *Ty-2* begomovirus resistance locus of tomato," *Theor. Appl. Genet.*, vol. 131, pp. 1345–1362, 2018.

[43] U. Gill *et al.*, "*Ty-6*, a major begomovirus resistance gene on chromosome 10, is effective against *Tomato yellow leaf curl virus* and *Tomato mottle virus*," *Theor. Appl. Genet.*, vol. 132, pp. 1543–1554, 2019.

[44] B. Mabvakure *et al.*, "Ongoing geographical spread of *Tomato yellow leaf curl virus*," *Virology*, vol. 498, pp. 257–264, 2016.

[45] E. Fiallo-Olivé and J. Navas-Castillo, "Begomoviruses: What is the secret(s) of their success?" *Trends Plant Sci.*, vol. 28, pp. 715–727, 2023.

[46] P. Roumagnac *et al.*, "Establishment of five new genera in the family *Geminiviridae*," *Arch. Virol.*, vol. 167, pp. 695–710, 2022.

[47] A. R. Horowitz, M. Ghanim, E. Roditakis, R. Nauen, and I. Ishaaya, "Insecticide resistance and its management in *Bemisia tabaci* species," *J. Pest Sci.*, vol. 93, pp. 893–910, 2020.

[48] R. A. C. Dângelo, M. Michereff-Filho, M. R. Campos, P. S. da Silva, and R. N. C. Guedes, "Insecticide resistance and control failure likelihood of the whitefly *Bemisia tabaci* (MEAM1; B biotype): A Neotropical scenario," *Ann. Appl. Biol.*, vol. 172, no. 1, pp. 88–99, 2018.

[49] S. Skendžić, M. Zovko, I. P. Živković, V. Lešić, and D. Lemić, "The impact of climate change on agricultural insect pests," *Insects*, vol. 12, no. 5, p. 440, 2021.

[50] Thesis, "Tomato - Yellow Leaf Curl Virus Dataset," Roboflow Universe, Roboflow, Dec. 2022. [Online]. Available: https://universe.roboflow.com/thesis-okplj/tomato-yellow-leaf-curl-virus

[51] mohammeda5, "Tomato Yellow Leaf Curl Virus Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/mohammeda5/tomato-yellow-leaf-curl-virus-dataset

ANNEXURE I: PLAGIARISM REPORT

[The institutional plagiarism-check report is to be attached here after the final text is submitted for checking.]